GLOBAL ACCOUNTING PLATFORM

Phase 1 Implementation Pack

Foundation • Architecture • Database • RBAC • Onboarding • API • Design System • Codex

Release strategy: all 14 phases will be completed before public V1. This document defines Phase 1 in build-ready detail.

Design direction: the web application will use a design-system approach similar to the current RetailFlow Web product—clean operational dashboards, restrained UI, efficient tables/forms, predictable navigation and reusable components—while maintaining a distinct brand identity and accounting-specific interaction patterns.

Version 1.0 • August 2026

1. Phase 1 Objective

Phase 1 establishes the product foundation that every later module depends on. The goal is not merely to create authentication and settings screens; it is to establish the tenancy, authorization, accounting primitives, localization model, auditability, engineering conventions and shared UI system that make later sales, purchasing, banking, inventory, projects and reporting modules safe to build.

> **Phase 1 is complete only when a user can create a secure organization, configure its accounting environment, invite a team, assign permissions, and enter the application with a valid ledger-ready organization context.**

Phase 1 deliverables

- Repository/application structure and engineering standards
- Authentication and session management
- Multi-tenant organization model
- Organization onboarding wizard
- Organization switcher
- Users, invitations, roles and permissions
- Country, currency, timezone and locale foundations
- Tax configuration foundation
- Fiscal year and accounting-period foundation
- Document numbering sequences
- Chart-of-accounts seed and system-account model
- Ledger schema and posting service skeleton
- Audit and security event infrastructure
- Global application shell and RetailFlow-inspired design system
- Admin/settings routes required for Phase 1
- API contract and validation patterns
- Seed data, testing strategy and demo tenant
- Master Codex implementation prompt

2. Design System Direction — RetailFlow Web Reference

> **Use RetailFlow Web as the current reference for design-system behavior and product feel. Do not copy its product terminology or branding; reuse the same quality bar, structural patterns and interaction philosophy.**

What should feel similar

Area	Direction
App shell	Persistent left navigation on desktop, compact responsive navigation on smaller screens, clear page headers and predictable content width.
Dashboard density	Operational and information-rich without feeling crowded; summary cards should support decisions rather than decoration.
Tables	High-quality data tables with search, filters, sorting, pagination, row actions, bulk actions and clear empty/loading states.
Forms	Clean grouped forms with strong labels, helper text, inline validation, sensible defaults and clear save/cancel hierarchy.
Actions	One obvious primary action per screen; secondary actions grouped consistently.
Status	Compact, reusable status badges/chips with semantic labels and icon support where useful.
Modals/drawers	Use drawers for fast contextual creation/editing; use full pages for complex financial documents.
Spacing	Consistent 4/8-based spacing rhythm, restrained card use and strong whitespace discipline.
Feedback	Toast + inline success/error feedback; destructive and accounting-impacting actions require explicit confirmation.
Responsiveness	Desktop-first finance workflows but fully usable on tablet and responsive mobile layouts.

Accounting-specific additions

- Transaction status header component
- Money and currency inputs with precision handling
- Tax selector and tax breakdown component
- Line-item editor with keyboard-friendly row entry
- Journal debit/credit grid
- Totals summary card
- Accounting impact preview
- Period-lock warning
- Exchange-rate component
- Audit/activity timeline
- Permission-aware action menu
- Approval status and approval-chain component

Initial token guidance

Token family	Guidance
Primary	Product brand primary to be finalized; components must rely on semantic tokens rather than raw hex values.
Surface	Neutral white/light surfaces with low-noise borders; avoid excessive colored cards.
Typography	Clear sans-serif UI type, strong numeric legibility, tabular numerals where possible.
Radius	Moderate, consistent radius; avoid overly playful rounded UI.
Elevation	Minimal shadows; prefer border + subtle surface separation.
Density	Comfortable by default; compact data-table density option can be introduced later.
Motion	Short, functional transitions only.

3. Phase 1 Technical Architecture

Layer	Decision
Frontend	Next.js + TypeScript
Backend	NestJS + TypeScript
Database	PostgreSQL
ORM	Prisma recommended for typed schema/migrations; critical ledger/reporting queries may use explicit SQL where justified
Cache/Jobs	Redis + BullMQ
Storage	S3-compatible object storage abstraction
Validation	Shared DTO/schema validation; reject unknown/untrusted fields
Auth	Secure email/password first; architecture should allow OAuth/SSO later
Observability	Structured logs, error tracking and OpenTelemetry-ready instrumentation
Deployment	Containerized environments: local, development, staging, production

Architecture style

Build Phase 1 as a modular monolith with strict domain boundaries. Avoid premature microservices. Each domain owns its rules and exposes services/events rather than allowing arbitrary cross-domain database writes.

Suggested monorepo

```
apps/
  web/                  # Next.js web application
  api/                  # NestJS API
packages/
  ui/                   # design-system components
  config/               # lint, tsconfig, env schemas
  contracts/            # shared API/domain types where appropriate
  accounting-core/      # accounting primitives, money, posting contracts
  localization/         # countries, currencies, locale metadata
  test-utils/
```

```
infrastructure/
  docker/
  migrations/
  scripts/
docs/
  architecture/
  adr/
  api/
```

Backend modules

```
src/modules/
  auth/
  users/
  organizations/
  memberships/
  rbac/
  localization/
  currencies/
  taxes/
  fiscal-periods/
  numbering/
  accounting/
  audit/
  notifications/
  platform/
```

4. Phase 1 Page Routes

Route	Purpose	Auth
/	Marketing/redirect entry	Public
/login	Sign in	Public
/signup	Create user account	Public
/verify-email	Email verification	Public/token
/forgot-password	Request reset	Public
/reset-password	Reset password	Public/token
/onboarding	Onboarding controller/stepper	Authenticated
/onboarding/organization	Business identity	Authenticated
/onboarding/localization	Country, currency, timezone, locale	Authenticated
/onboarding/accounting	Fiscal year + accounting defaults	Authenticated
/onboarding/tax	Tax registration/configuration	Authenticated
/onboarding/numbering	Document numbering defaults	Authenticated
/onboarding/team	Invite team / skip	Authenticated
/app	Organization-aware app redirect	Authenticated/member
/app/[org]/dashboard	Phase 1 foundation dashboard	Authenticated/member
/app/[org]/settings/organization	Organization profile	Authorized
/app/[org]/settings/users	Members/invitations	Authorized
/app/[org]/settings/roles	Roles and permissions	Authorized
/app/[org]/settings/currencies	Currency settings	Authorized
/app/[org]/settings/taxes	Tax settings	Authorized
/app/[org]/settings/fiscal-periods	Fiscal year/period controls	Authorized
/app/[org]/settings/numbering	Number sequences	Authorized
/app/[org]/settings/chart-of-accounts	Seed/account foundation	Authorized
/app/[org]/settings/audit-log	Audit history	Authorized
/app/[org]/settings/security	Security/session controls	Authorized

5. Onboarding Flow

The onboarding should visually follow the same structured, low-friction style as RetailFlow Web: a clear progress stepper, focused single-purpose panels, sensible defaults and the ability to save progress. Avoid long, intimidating accounting forms.

Step	Required fields	Rules
1. Account	Name, verified email, password/auth method	Email verification before organization finalization
2. Organization	Legal/business name, trading name optional, business type, country	Country is required because it controls defaults and compliance messaging
3. Localization	Base currency, timezone, locale/language	Currency defaulted from country but editable before

		posting activity
4. Accounting	Fiscal year start, accounting basis preference, chart template	Accrual ledger foundation remains canonical; preference may affect reporting views later
5. Tax	Registered/not registered, tax regime, tax ID, tax-inclusive default	Only show jurisdiction options supported by active country pack
6. Numbering	Invoice, quote, sales order, credit note, bill/PO sequence defaults	Preview resulting document numbers
7. Team	Invite emails + role or skip	Owner role automatically assigned to creator
8. Review	Summary, edit links, confirm organization	Creates foundation seed transaction/config bundle atomically

Onboarding completion actions

- Create organization record
- Create owner membership
- Create default roles and role-permission bindings
- Create base currency and enabled currency record
- Create fiscal year and accounting periods
- Seed default chart of accounts based on selected template/country
- Create system/control accounts
- Create default tax codes/rates if selected
- Create number sequences
- Create default organization preferences
- Emit organization.created and onboarding.completed events
- Write audit events for material configuration

6. Organization Settings Specification

Section	Fields
Profile	Legal name, trading name, registration number, tax ID(s), website, email, phone, logo, industry, business type
Address	Country, address lines, city, region/state, postal code
Localization	Locale/language, timezone, date format display preference, first day of week
Accounting	Base currency, fiscal year start, rounding account, retained earnings account, exchange gain/loss accounts
Documents	Default payment terms, notes/terms defaults, numbering, template placeholders
Tax	Registration status, tax mode, default tax behavior, tax IDs
Security	Session visibility, MFA readiness flag, sensitive-action confirmation policy

Protected settings

- Base currency cannot be casually changed after posted accounting activity.
- Fiscal-year structure changes after activity require controlled migration rules.
- System/control accounts cannot be deleted.
- Country-pack changes must not retroactively rewrite finalized transaction snapshots.
- Number sequence values already issued cannot be reused.

7. RBAC Permission Keys

Permissions should use stable machine-readable keys. The frontend may group them into friendly labels, but authorization is enforced in the API.

Group	Permission keys
organization	organization.view organization.update organization.delete organization.transfer_ownership
members	members.view members.invite members.update

	members.remove members.resend_invite
roles	roles.view roles.create roles.update roles.delete roles.assign
settings	settings.localization.manage settings.currency.manage settings.tax.manage settings.fiscal.manage settings.numbering.manage settings.accounting.manage
accounts	accounts.view accounts.create accounts.update accounts.deactivate accounts.opening_balances.manage
journals	journals.view journals.create journals.post journals.reverse journals.approve
audit	audit.view audit.export
security	security.view security.sessions.manage security.mfa.manage
platform	platform.support_access platform.feature_flags.manage

Default role baseline

Role	Phase 1 permissions
Owner	All organization-scoped permissions; ownership transfer protected
Administrator	All operational settings/users/roles except ownership transfer and selected destructive controls
Accountant	Accounting settings, accounts, journals, fiscal periods, audit read/export
Sales	Minimal Phase 1 access; organization read only until Sales module arrives
Purchases	Minimal Phase 1 access; organization read only until Purchases module arrives
Inventory Manager	Organization read + later inventory rights
Project Manager	Organization read + later project rights
Viewer/Auditor	Read-only organization and allowed audit/report views

8. Phase 1 Database Schema

The following schema is the minimum Phase 1 relational model. Exact column names may be adjusted during implementation, but the invariants should not change.

Entity	Key columns / constraints
users	id UUID PK; email CITEXT unique; email_verified_at; display_name; password_hash nullable; status; created_at; updated_at
organizations	id UUID PK; slug unique; legal_name; trading_name; country_code; base_currency_code; timezone; locale; status; onboarding_status; created_by_user_id; created_at; updated_at
organization_members	id; organization_id FK; user_id FK; role_id FK; status; joined_at; UNIQUE(org,user)
organization_invitations	id; organization_id; email; role_id; token_hash; expires_at; invited_by; accepted_at; status
roles	id; organization_id nullable for system templates; key; name; description; is_system; is_owner_role
permissions	id; key unique; description; module
role_permissions	role_id; permission_id; UNIQUE(role,permission)
countries	code PK; name; default_currency_code; default_timezone nullable; status; metadata_json
currencies	code PK; name; symbol; decimals; status
organization_currencies	organization_id; currency_code; is_base; enabled; created_at
exchange_rates	id; organization_id nullable; base_currency; quote_currency; rate decimal; rate_date; source; created_at
tax_regimes	id; country_code; key; name; version; status; metadata_json
tax_rates	id; tax_regime_id; code; name; rate decimal; type; effective_from; effective_to; active
organization_tax_settings	organization_id PK; tax_regime_id nullable; registration_status; tax_id;

	prices_include_tax; metadata_json
fiscal_years	id; organization_id; name; start_date; end_date; status
accounting_periods	id; organization_id; fiscal_year_id; start_date; end_date; status OPEN/SOFT_CLOSED/LOCKED; locked_at; locked_by
number_sequences	id; organization_id; document_type; prefix; next_number bigint; padding; suffix nullable; reset_policy; UNIQUE(org,document_type)
accounts	id; organization_id; code; name; account_type; subtype; parent_id nullable; currency_code nullable; system_key nullable; is_control; active; UNIQUE(org,code)
journal_entries	id; organization_id; entry_no; posting_date; source_type; source_id; memo; status DRAFT/POSTED/REVERSED; currency_code; reversal_of_id nullable; posted_at; posted_by
journal_lines	id; journal_entry_id; account_id; contact_id nullable; debit decimal; credit decimal; currency_code; foreign_amount nullable; exchange_rate nullable; memo; dimensions_json
audit_events	id; organization_id nullable; actor_user_id nullable; event_key; entity_type; entity_id; action; before_json; after_json; metadata_json; occurred_at
security_events	id; user_id nullable; organization_id nullable; event_key; severity; metadata_json; occurred_at

9. Accounting Foundation

Even though Phase 1 has no full invoice or bill UI, the ledger must already be designed and testable. Later modules must call a posting service rather than writing journal rows directly.

Core invariants

- For every POSTED journal entry: sum(debit) = sum(credit).
- A journal line cannot contain both positive debit and positive credit.
- A posted journal entry is immutable; reversal creates a new balanced entry.
- Posting date must fall in an open accounting period.
- Organization base currency is present on every journal entry and reporting amount.
- Source type + source ID should be unique for idempotent system postings where applicable.
- Journal numbers are generated from an organization sequence and are not reused.
- Control/system accounts cannot be removed while the organization exists.
- Account deactivation affects future selection, not historical reports.

System account keys

System key	Purpose
accounts_receivable	Customer control account
accounts_payable	Vendor control account
sales_revenue	Default revenue
general_expense	Default expense
bank_default	Default bank/cash placeholder
tax_payable	Output tax/VAT/GST liability
tax_receivable	Input/recoverable tax
inventory_asset	Inventory control
cogs	Cost of goods sold
retained_earnings	Equity closing account
fx_gain	Foreign exchange gains
fx_loss	Foreign exchange losses
rounding	Rounding differences

Posting service contract

```
PostingRequest {
  organizationId
  sourceType
  sourceId
  postingDate
  memo
  transactionCurrency
  exchangeRate?
  lines: [
```

```
    {
      accountId
      debit?
      credit?
      foreignAmount?
      contactId?
      dimensions?
      memo?
    }
  ]
  idempotencyKey
}

PostingResult {
  journalEntryId
  entryNo
  status
  postedAt
}
```

10. Phase 1 API Endpoints

Method	Endpoint	Purpose
POST	/auth/signup	Create user
POST	/auth/login	Authenticate
POST	/auth/logout	End session
POST	/auth/verify-email	Verify token
POST	/auth/forgot-password	Request reset
POST	/auth/reset-password	Reset password
GET	/me	Current user/profile
GET	/organizations	Organizations current user can access
POST	/organizations	Create organization/onboarding draft
GET	/organizations/:orgId	Organization detail
PATCH	/organizations/:orgId	Update permitted organization fields
POST	/organizations/:orgId/complete-onboarding	Atomically finalize foundation setup
GET	/organizations/:orgId/members	List members
POST	/organizations/:orgId/invitations	Invite member
PATCH	/organizations/:orgId/members/:memberId	Change role/status
DELETE	/organizations/:orgId/members/:memberId	Remove member
GET	/organizations/:orgId/roles	List roles
POST	/organizations/:orgId/roles	Create custom role
PATCH	/organizations/:orgId/roles/:roleId	Update role
GET	/permissions	Permission catalogue
GET	/localization/countries	Supported countries
GET	/localization/currencies	Currencies
GET	/organizations/:orgId/currencies	Organization currencies
POST	/organizations/:orgId/currencies	Enable currency
GET	/organizations/:orgId/taxes	Tax configuration
PATCH	/organizations/:orgId/taxes	Update tax configuration
GET	/organizations/:orgId/fiscal-years	Fiscal years/periods
POST	/organizations/:orgId/fiscal-years	Create fiscal year when allowed
PATCH	/organizations/:orgId/periods/:periodId	Soft close/lock/unlock
GET	/organizations/:orgId/number-sequences	Sequences
PATCH	/organizations/:orgId/number-sequences/:type	Update sequence configuration
GET	/organizations/:orgId/accounts	Chart of accounts
POST	/organizations/:orgId/accounts	Create account
PATCH	/organizations/:orgId/accounts/:accountId	Update/deactivate account
POST	/organizations/:orgId/journals	Create draft manual journal
POST	/organizations/:orgId/journals/:journalId/post	Post journal
POST	/organizations/:orgId/journals/:journalId/reverse	Reverse journal
GET	/organizations/:orgId/audit-events	Audit log

API requirements

- Organization context validated from authenticated membership; never trust a client-supplied organization ID without authorization.
- Consistent error envelope with machine-readable code, user-safe message and field errors.
- Pagination uses cursor or stable page strategy consistently.
- All financial/config mutations accept idempotency where duplicate submission is plausible.

- Authorization checked in service/controller guards and backed by domain rules.
- Audit event emitted after successful material mutation.
- Use optimistic concurrency/versioning on settings where conflicting admin edits could matter.

11. Phase 1 Shared UI Components

Component	Purpose
AppShell	RetailFlow-like left navigation, top header, organization context and responsive behavior
OrganizationSwitcher	Switch between organizations without reauthentication
PageHeader	Breadcrumb/title/subtitle/primary action/action menu
DataTable	Search/filter/sort/pagination/column controls/row actions/empty state
FormSection	Consistent grouped settings forms
SettingsNav	Stable settings information architecture
StatusBadge	Reusable semantic statuses
MoneyInput	Decimal + currency-aware display/validation
CurrencySelect	Currency code/name/symbol display
CountrySelect	Country and localization defaults
DateInput	Locale-aware presentation, canonical date value
PermissionGate	Hide/disable UI actions based on authorization
ConfirmDialog	Destructive or material actions
AccountingWarning	Period-lock/system-account/config impact warning
AuditTimeline	Activity/audit events
Stepper	Onboarding wizard
EmptyState	Action-oriented no-data experience
Toast/InlineAlert	Success/warning/error feedback

12. Validation, Security & Audit Rules

Area	Rule
Email	Normalize and validate; unique user identity strategy defined
Slugs	Unique, URL-safe; changes controlled if routes depend on slug
Currency	ISO-style currency code; precision from currency table
Money	Decimal; max/min validated at domain boundary
Dates	Financial posting date is date-only; audit timestamps are UTC instants
Passwords	Modern adaptive password hashing; no plaintext logs
Sessions	HttpOnly/Secure/SameSite cookies or equivalent secure token strategy
Invites	Store token hash, not raw invitation token
Tenant isolation	Every organization query scoped and tested
Privilege change	Audit actor, old role/permissions and new role/permissions
Period unlock	Requires authorized permission + reason + audit event
System account change	Restricted and audited

13. Phase 1 Test Plan

Automated tests

- Unit tests for money/rounding, numbering, permission checks, period-lock rules and journal balancing.
- Integration tests for organization onboarding transaction.
- Integration tests for role assignment and forbidden access.
- Integration tests for journal post/reverse/idempotency.
- Database tests proving unique membership and number-sequence constraints.
- Cross-tenant API tests for every organization-scoped endpoint.
- UI tests for onboarding, organization switch, invite user and settings save.
- Accessibility checks on authentication, onboarding and settings flows.

Golden Phase 1 scenarios

1. New user signs up, verifies email, completes organization onboarding and reaches the dashboard.

2. Organization defaults are seeded atomically: periods, accounts, tax config and numbering.
3. Owner invites an accountant; accountant accepts and sees only authorized settings.
4. Administrator creates a custom role; API blocks a permission not granted to the administrator where privilege escalation protection applies.
5. Accountant posts a balanced manual journal; unbalanced journal is rejected.
6. Journal posting is retried with the same idempotency key and does not duplicate.
7. Period is locked; backdated journal posting fails; authorized owner unlocks with reason; event is audited.
8. User from Organization A cannot access Organization B by changing route/API identifiers.
9. Base currency change is blocked after posted financial activity.

14. Phase 1 Engineering Milestones

Milestone	Deliverable	Exit gate
1A — Workspace	Monorepo, environments, CI, lint/test/build, env validation	Web/API build green in CI
1B — Identity	Signup/login/verify/reset/session	Security tests + basic UI complete
1C — Tenancy	Organizations, membership, switcher, tenant guards	Cross-tenant tests pass
1D — RBAC	Permission catalogue, roles, guards, UI gates	Privilege tests pass
1E — Localization	Countries, currencies, locale/timezone foundations	Onboarding defaults work
1F — Accounting Setup	Fiscal years/periods, chart template, system accounts, numbering	Organization seed atomic
1G — Ledger Core	Journal schema, post/reverse, balancing/idempotency	Golden journal tests pass
1H — Tax Foundation	Regime/rate/config models	Country-pack tax defaults seed correctly
1I — Settings UI	Org/users/roles/currency/tax/fiscal/numbering/accounts/audit	Responsive UX complete
1J — Hardening	Audit/security events, observability, seed/demo org, docs	Phase 1 acceptance suite green

15. Phase 1 Dashboard

The dashboard is intentionally limited before sales/purchases arrive. It should feel complete but not fabricate business metrics.

Widget	Phase 1 content
Setup Progress	Organization profile, tax, numbering, team, chart/accounts readiness
Organization Summary	Country, base currency, fiscal year, timezone
Team	Active members, pending invitations
Accounting Status	Open period, chart of accounts count, any locked period warning
Quick Actions	Invite user, edit tax settings, review accounts, configure numbering
Recent Activity	Audit events relevant to setup/security

As later phases ship internally, this dashboard evolves into the full business overview. Do not design throwaway dashboard components; use the shared card/table primitives from the RetailFlow-inspired system.

16. Master Codex Prompt — Phase 1

Use the following as the initial master implementation prompt. It is deliberately specific about sequencing and invariants so Codex does not create disconnected screens.

```
You are a senior full-stack SaaS architect and engineer. Build Phase 1 (Foundation) of a global multi-tenant
accounting and invoicing web application.

PRODUCT CONTEXT
- The final V1 will be a complete global accounting/business-finance product similar in capability breadth to Zoho
Books.
- All 14 product phases must be completed before public V1.
- Online payment gateway integrations are NOT part of V1. Do not add Stripe, PayPal, Paystack, Flutterwave, M-Pesa
or any provider SDK.
- Manual payment recording will be added in later phases and must eventually post into the ledger.
- Phase 1 must establish a production-quality foundation for all later modules.

DESIGN SYSTEM
- Use a design-system approach similar to the current RetailFlow Web application.
- Match its general product feel: clean operational UI, structured left navigation, strong page headers, efficient
data tables, restrained cards, clear forms, consistent status chips, drawers for lightweight contextual tasks,
full pages for complex workflows, strong empty/loading/error states and responsive behavior.
```

```
- Do NOT copy RetailFlow branding, wording or business-specific modules.
- Build reusable components in packages/ui and use semantic design tokens so branding can change independently.
- Add accounting-specific components: MoneyInput, CurrencySelect, AccountingWarning, AuditTimeline, PermissionGate
and onboarding Stepper.
- Prioritize desktop finance workflows while keeping tablet/mobile responsive.

TECH STACK
- Next.js + TypeScript web
- NestJS + TypeScript API
- PostgreSQL
- Prisma
- Redis + BullMQ
- S3-compatible storage abstraction
- Containerized environments
- Structured logging and OpenTelemetry-ready instrumentation
- Modular monolith architecture, not microservices

MONOREPO
apps/web
apps/api
packages/ui
packages/config
packages/contracts
packages/accounting-core
packages/localization
packages/test-utils
infrastructure/
docs/

CORE DOMAINS
auth, users, organizations, memberships, rbac, localization, currencies, taxes, fiscal-periods, numbering,
accounting, audit, notifications, platform.

PHASE 1 REQUIRED FLOWS
1. Sign up
2. Verify email
3. Sign in/out
4. Forgot/reset password
5. Create organization
6. Complete onboarding wizard:
   - organization identity
   - country/base currency/timezone/locale
   - fiscal year/accounting setup
   - tax setup
   - document numbering
   - team invitations
   - final review
7. Seed organization defaults atomically
8. Organization switcher
9. Users/invitations
10. Roles/permissions
11. Currency settings
12. Tax settings
13. Fiscal years/accounting periods
14. Number sequences
15. Chart of accounts
16. Audit log
17. Manual journal create/post/reverse foundation
18. Foundation dashboard

TENANCY
- All organization-owned tables must include organization_id.
- Never authorize based only on an org ID from the client.
- Resolve access through authenticated membership.
- Cross-tenant access is a release-blocking security bug.
- Add automated cross-tenant tests for every organization-scoped API.

RBAC
Use stable permission keys including:
organization.view
organization.update
organization.delete
organization.transfer_ownership
members.view
members.invite
members.update
```

```
members.remove
members.resend_invite
roles.view
roles.create
roles.update
roles.delete
roles.assign
settings.localization.manage
settings.currency.manage
settings.tax.manage
settings.fiscal.manage
settings.numbering.manage
settings.accounting.manage
accounts.view
accounts.create
accounts.update
accounts.deactivate
accounts.opening_balances.manage
journals.view
journals.create
journals.post
journals.reverse
journals.approve
audit.view
audit.export
security.view
security.sessions.manage
security.mfa.manage

DATABASE
Implement:
users
organizations
organization_members
organization_invitations
roles
permissions
role_permissions
countries
currencies
organization_currencies
exchange_rates
tax_regimes
tax_rates
organization_tax_settings
fiscal_years
accounting_periods
number_sequences
accounts
journal_entries
journal_lines
audit_events
security_events

ACCOUNTING INVARIANTS
- Money uses fixed-precision decimal types. Never use binary floating point.
- Posted journal entries must balance exactly.
- A line cannot carry both a positive debit and positive credit.
- Posted entries are immutable.
- Corrections use reversal entries.
- Posting date must be inside an OPEN period.
- Every posting is organization-scoped.
- System/control accounts cannot be deleted.
- Base currency is protected after posted financial activity.
- Posting must support idempotency keys.
- Use a PostingService. Later modules must never write journal lines directly.

SEED SYSTEM ACCOUNTS
accounts_receivable
accounts_payable
sales_revenue
general_expense
bank_default
tax_payable
tax_receivable
inventory_asset
```

```
cogs
retained_earnings
fx_gain
fx_loss
rounding

API
Implement versioned REST endpoints for auth, organizations, members, roles, permissions, localization, currencies,
taxes, fiscal years/periods, number sequences, accounts, manual journals and audit events. Use DTO/schema
validation and a consistent error envelope.

ROUTES
/login
/signup
/verify-email
/forgot-password
/reset-password
/onboarding/*
/app/[org]/dashboard
/app/[org]/settings/organization
/app/[org]/settings/users
/app/[org]/settings/roles
/app/[org]/settings/currencies
/app/[org]/settings/taxes
/app/[org]/settings/fiscal-periods
/app/[org]/settings/numbering
/app/[org]/settings/chart-of-accounts
/app/[org]/settings/audit-log
/app/[org]/settings/security

AUDIT
Record actor, organization, event key, entity type/id, action, before/after material values, metadata and
timestamp for important changes. Tenant users must not be able to mutate audit history.

SECURITY
- Secure password hashing
- HttpOnly/Secure/SameSite session strategy
- Invitation token hashes, never raw tokens at rest
- Rate-limit authentication and sensitive endpoints
- Validate all input
- Prevent privilege escalation
- Protect period unlock, ownership transfer and system-account changes
- Do not leak whether inaccessible cross-tenant records exist

TESTS
Add unit, integration and end-to-end tests.
Required golden tests:
- Complete onboarding seeds organization atomically
- Owner invite → accountant acceptance
- Custom role authorization
- Balanced journal posts
- Unbalanced journal rejected
- Idempotent journal retry creates no duplicate
- Locked period rejects posting
- Authorized unlock records reason/audit
- Cross-tenant access denied
- Base currency change blocked after posted activity

IMPLEMENTATION ORDER
1. Workspace/CI
2. Identity
3. Tenancy
4. RBAC
5. Localization/currency
6. Fiscal/accounting setup
7. Ledger core
8. Tax foundation
9. Settings UI
10. Audit/security/observability/hardening

ENGINEERING REQUIREMENTS
- Do not generate placeholder architecture that later has to be discarded.
- Avoid giant files and domain leakage.
- Use migrations and seeds.
- Write ADRs for important architectural choices.
- Add a README explaining local setup.
```

```
- Create demo seed data.
- Keep UI components reusable and aligned to the RetailFlow-inspired design system.
- Do not start Phase 2 Sales until all Phase 1 acceptance tests pass.

Before coding each milestone, inspect the existing repository, state what you will change, and preserve existing
conventions unless they conflict with these requirements.
```

17. Phase 1 Definition of Done

Category	Done when
Design	RetailFlow-inspired design-system primitives are documented and used consistently in all Phase 1 screens.
Auth	Signup/login/verify/reset/session flows work and are hardened.
Tenancy	Organization switching and tenant isolation pass automated tests.
RBAC	Default/custom roles and API enforcement work without privilege escalation.
Onboarding	A new organization is fully seeded atomically and can enter the application.
Localization	Country/currency/timezone/locale model works and can support future country packs.
Accounting	Chart/system accounts, periods, numbering and journal posting/reversal are functional.
Tax	Tax regime/settings foundation exists without hard-coded country logic.
Audit	Material changes and security events are recorded and viewable by authorized users.
UI	All settings pages have responsive, loading, empty, error and permission-denied states.
Testing	Golden Phase 1 scenarios and cross-tenant tests are green.
Docs	Architecture decisions, API, environment setup and seed strategy are documented.

Do not move to Phase 2 simply because the screens look complete. Phase 1 is accepted only when tenancy, authorization, accounting invariants and setup seeding are demonstrably correct.